A BM MANUAL

# bm: the bootloader and the operating system

What bm is, how it starts, what its menu and its keys do, and which files it keeps on the SD card.

# Contents


**What bm is** . . . . . **4**

What it does . . . . . 4

Good to know . . . . . 4

**The boot chain** . . . . . **5**

On the Pi Zero W . . . . . 5

Loading a kernel from the PC . . . . . 5

On the RGB30 . . . . . 6

**What the kernel does at start** . . . . . **7**

**The menu and its tabs** . . . . . **8**

The menu's keys . . . . . 8

Good to know . . . . . 8

Settings . . . . . 8

**The system keys** . . . . . **10**

Good to know . . . . . 10

**Find out the keys of a game** . . . . . **11**

**The monitor** . . . . . **12**

How to open it . . . . . 12

Commands . . . . . 12

A line of commands . . . . . 13

**The files on the SD card** . . . . . **14**

**Settings and config.txt** . . . . . **15**

Change a setting from the PC . . . . . 15

**Screenshots** . . . . . **17**

Good to know . . . . . 17

Try it, step by step . . . . . 17

**Saves** . . . . . **18**

**Crashes and logs** . . . . . **19**

A crash . . . . . 19

Where the logs are . . . . . 19

**Updating** . . . . . **20**

From the console . . . . . 20

From the PC . . . . . 20

**Open points** . . . . . **21**

README

**What this is.** A guided tour of bm itself: the boot, the kernel, the menu, the system keys, the monitor and the files on the card. It is written for a curious user or developer.

**How to use it.** Read chapter 1 for the idea, then jump to what you need. Every how-to is a list of numbered steps; anything in a bordered box with a shadow is meant to be copied as it is.

**What is missing.** Every fact comes from the repository. Where something was not written down or not tried on the hardware, the manual shows a highlighted TO FILL tag; nothing is invented.

# What bm is

bm (BareMetal) is a games console and a development environment that runs on the **Raspberry Pi Zero W** and on the **PowKiddy RGB30**. It has no operating system underneath: the repository describes it as a bare-metal kernel with no Linux, no background daemons and no scheduler of an OS. The kernel talks to the hardware registers itself.

| Board | Chip | Image the card carries | Build |
|---|---|---|---|
| Pi Zero W (also Pi 1) | BCM2835, ARM1176 | `kernel.img` | `make` |
| PowKiddy RGB30 | RK3566, Cortex-A55 (AArch64) | `kernel8.img` | `make TARGET=rgb30` |

## What it does

The kernel starts, shows a menu, and runs **cartridges**: single files ( `.bm` , and `.b16` on the RGB30) that hold the Lua 5.4 code, sprites, maps, 3D meshes, animations and sounds of a game. The same menu also opens the editors of the suite (the **Dev** tab), the Market and the Settings.

## Good to know

- One menu serves both boards ( `src/kernel/menu_ui.c` ): on the Pi it is 640x360, on the RGB30 it is 360x360 shown twice as large.
- The repository says the Pi Zero 2 W code stays in the tree but is switched off in the normal build.
- A `.b16` never replaces a `.bm` : they are two separate editions of a game.

# The boot chain

## On the Pi Zero W

1. You switch on. The Pi's own firmware reads the FAT32 card. It needs `bootcode.bin`, `start.elf` and `fixup.dat` in the root of the card.
2. The firmware reads `config.txt` in the root (the firmware's file, not bm's: see chapter 9) and loads the file named by `kernel=`, which is `kernel.img`, at address 0x8000.
3. bm's own code takes over: only core 0 runs, the other cores are parked, the stacks are set up and the working memory is cleared.
4. The kernel starts (next chapter) and ends in the menu.

The `config.txt` that bm ships asks the firmware for these things:

| Line | What it does |
|---|---|
| `kernel=kernel.img` | the BCM2835 boards (Pi Zero, Zero W, Pi 1) start this file |
| `hdmi_force_hotplug=1` | always send HDMI, even if no monitor is detected |
| `hdmi_drive=2` | HDMI mode (not DVI), so the TV gets audio |
| `disable_overscan=1` | use the full screen area |
| `scaling_kernel=8` | the GPU enlarges the 640x360 picture with sharp pixels |
| `hdmi_group=1` and `hdmi_mode=16` | always 1080p60, so a 16:9 picture keeps its shape |
| `disable_splash=1` | skip the rainbow splash |
| `enable_uart=1` | keeps the core clock stable for the serial console |

## Loading a kernel from the PC

The repository also builds a small **chainloader** (`chainloader/`, built by `make` as `build/chainloader.img`). Put on the card in place of the kernel, it waits on the serial port, receives a `kernel.img` from the PC, checks its CRC and jumps to it at 0x8000. While it waits it blinks the LED and sends three bytes every second.

> **NOTE**
> The monitor command `r` reboots the Pi, and the chainloader then asks for a new kernel. That is how a developer tries a build without touching the card.

> **COPY AND PASTE**
> ```
> make sdcard-chainloader
> make run-serial
> ```

`make sdcard-chainloader` prepares the card contents with the chainloader as the kernel. `make run-serial` sends `build/kernel.img` with `tools/bm_load.py`, then opens a terminal. The serial line is 115200 baud, 8N1, on GPIO14 and GPIO15.

## On the RGB30

The RGB30 does not use the Pi's firmware. The order is: the chip's boot ROM, then `idbloader.img` (memory set-up), then `u-boot.itb` (U-Boot), then `/extlinux/extlinux.conf` on the first partition of the card in slot TF1, which starts `kernel8.img`. The bootloader files come from the official ROCKNIX image and sit before the partition, so you must not format or repartition the card. Power on with the **power button**.

# What the kernel does at start

On the Pi, `main.c` does these things in this order:

1. Starts the LED and the serial port, then the screen and the splash (the logo); the boot's lines go to the console.
2. Sets up the memory: the heap, the ARM clock at the maximum the firmware offers, and the MMU.
3. Detects the board and prints it with its SoC and revision code.
4. Sets up the board's LED, then the interrupts and the timer.
5. Tries to arm the watchdog (3 seconds). The boot log says `watchdog: on, 3000 ms` when it is armed. If the check fails, it says "not available, no freeze guard" (in an emulator: "resets at once on RSTC (emulator), no freeze guard"). The Pi Zero W is armed; this is not verified on a real console by this manual.
6. Starts the audio, the DMA and USB.
7. Loads the cartridges list and `bm/config.txt`.
8. Turns the crash report of the previous boot into a report (chapter 12).
9. Starts Bluetooth if a pad is paired, then WiFi and Ethernet.
10. Opens the cartridge menu. The monitor is the text console that runs after it.

The README says the Pi boots into the menu in about two seconds. **[TO FILL: the boot time of the RGB30 has not been measured in the repository]**

On the RGB30 a LED tells you how the boot is going when you have no serial cable:

| LED | Meaning |
|---|---|
| red on | booting; if it stays like this, the kernel hung early |
| solid green | bm is running and nothing is wrong |
| green blinking slowly | something else: boot not finished, the SD missing or unreadable, battery low, a kernel arriving, an update being installed |
| solid red and green blinking | the screen had problems: read `bm/bootlog.txt` |
| red blinking N times, pause | fatal exception N (1 synchronous, 4 SError, 9 panic) |

# The menu and its tabs

The menu is the first thing you see. It opens on **Games**.

| Tab | What it holds |
|---|---|
| Market | the catalog of games and apps you can download; two pages, **Games** and **Apps** |
| Games | the cartridges on the card |
| Dev | the tools: SDK, editors, Sound, Studio, Animator, Mesh, Pixel |
| Lib | the shared resources; hidden until you set `lib_tab=1` |
| Settings | the last tab: its panel opens in place of a list |

The Market loads only while its tab is on and never blocks the menu.

**[TO FILL: shots/boot-os-menu.png]**

## The menu's keys

| Key | What it does |
|---|---|
| arrows or `w` `a` `s` `d` | choose |
| `Enter` | play, open, yes |
| `c` | options of the game |
| `q` / `e` | the tab before, after |
| `Tab` | the next tab |
| `1` - `5` | Market ... Settings |
| `r` | read the SD card again |
| mouse | click plays; right click goes back |

Game options ( `c` ) show the bytes of the saves and a *Delete the save data* row.

## Good to know

- On the RGB30, **B** confirms and **A** goes back. Set `confirm=a` in `bm/config.txt` to swap them, or use Settings > Controllers > *Confirm button*.
- On the RGB30 the top bar shows only WiFi and battery. Games lists `.b16` files, and shows `.bm` files for testing: `show_bm=0` hides them. The Market lists only `.b16` : `market_bm=1` adds the `.bm` .
- The editors work on projects only. A `.bm` is read-only; Ctrl+B in an editor builds the final `.bm` .

## Settings

Settings has these pages: Controllers, WiFi and network (Network on a board without radio), Screen and sound, Updates, Reports and System, then the rows *Restart* and *Shut down*. System shows the version, the

board, the uptime, the memory in use, the CPU, the SD card and the battery, and has *Log since boot* and *Open the monitor*.

**[TO FILL: shots/boot-os-settings.png]**

# The system keys

The system keys work in the menu, inside games and inside the tools. Most are defined in one table, `src/kernel/syskeys.c`, which lists the keys shown on screen while you hold F12. Print Screen is not in that table: games and apps handle it separately (`src/bm/runtime.c`).

| Key | What it does |
|---|---|
| `F12` (hold) | shows the keys |
| `Esc` | the menu, or back; inside a game it is Start (the game's own menu) |
| `Ctrl+Esc` | back to bm's menu; the game stays suspended |
| `Ctrl+Shift+Esc` | the monitor |
| `F11` | performance overlay: off, top, bottom |
| `F1 - F4` | the pages (tools) |
| `F5` / `Ctrl+R` | try the game (tools) |
| `F6` | the assistant (tools) |
| `Ctrl+S`, `Ctrl+Shift+S`, `Ctrl+O`, `Ctrl+N` | save, save as, open, new (tools) |
| `Ctrl+Z`, `Ctrl+Y`, `Ctrl+X`, `Ctrl+C`, `Ctrl+V`, `Ctrl+F` | undo, redo, cut, copy, paste, find (tools) |
| `Print Screen` | saves a screenshot (chapter 10) |

On a pad, **PS** or **Start+Select** do what Ctrl+Esc does. The system marks F12, Ctrl+Esc, Ctrl+Shift+Esc and F11 as its own keys. A game that is suspended can be resumed from its cover.

## Good to know

- In a network match, the system does not suspend the game: it asks "Leave the match?" and, on yes, the game is closed.
- The overlay shows frames a second, the milliseconds of work in a frame and the CPU and GPU load. It lasts one run; every game starts as Settings > Screen and sound > *Performance overlay* says (off by default).
- From the serial line, Ctrl+\ works as Ctrl+Esc.

# Find out the keys of a game

Every game has its own keys, and F12 lists them next to the system's.

1. Start the console with a USB or Bluetooth keyboard connected. You should see the menu on the Games tab.
2. Choose a game with the arrow keys. The name of the game appears on a pill under the tabs.
3. Press `Enter`. The game starts.
4. Press and hold `F12`. A full-screen list titled `Keys` with `(F12 held)` appears, with the system keys first.
5. Read the game's own keys in the list.
6. Release `F12`. The list disappears and the game goes on.
7. Press `Ctrl+Esc` to leave. You are back in the menu; the game is suspended, and its cover now says Resume.

**[TO FILL: shots/boot-os-keys.png]**

TRY IT

Do the same in the menu itself: hold F12 on the Games tab. You get two groups, `bm` (the system keys) and `Menu` (the menu's keys).

NOTE

If the list does not show a game's keys, the game did not declare any. **[TO FILL: how to see the keys of a game when only a pad is connected; F12 is a keyboard key]**

# The monitor

The monitor is bm's text console. It has a prompt `>`, takes one key per command, and prints on the screen, on the serial port and on the network console.

## How to open it

- Press `Ctrl+Shift+Esc` anywhere.
- Or choose Settings > System > *Open the monitor*.
- Or connect from the PC: `python3 tools/bm_net.py IP` shows the monitor of a console on the network.

Type `h` for help. `M` goes back to the menu; `Esc` alone does nothing at the prompt.

## Commands

The help of the monitor lists many keys. These are the ones that matter to a first-time user:

| Key | What it does |
|---|---|
| `h` or `?` | the help |
| `M` | the cartridge menu |
| `f` / `F` | list cartridges / read the SD card again |
| `i` | system information |
| `m` | heap usage |
| `c` | clear the screen |
| `l` | Lua prompt (`Esc`, `Ctrl-D` or `exit()` returns) |
| `L` | keyboard layout, Italian or US |
| `V` | `.bm` drawing, direct on screen or via RAM |
| `y` / `Y` | USB scan / input test |
| `T` `K` `O` | Bluetooth: pair a pad, a keyboard, a mouse |
| `P` | Bluetooth: forget all paired devices (asks first) |
| `W` | WiFi: start, list networks, join one |
| `u` | update from GitHub (asks first) |
| `o` | everything printed since boot |
| `Z` | the log since boot, as a report |
| `r` | reboot |
| `X` | crash tests: then `u`, `a`, `b`, `s` or `f` |

Other keys run tests and benchmarks: `k`, `p`, `D`, `g`, `j`, `R`, `s`, `d`, `t`, `a`, `b`. Others open the editors: `e`, `A`, `3`, `4`, `5`, `6`, `C`, `I`.

## A line of commands

Type `:` and then commands separated by `;`. They run one after the other, and the line stops at the first one that fails.

| Command | What it does |
|---|---|
| `gpu` | the GPU test and its report |
| `b3d tests=a,b profiles=P` | the 3D Bench, all or part |
| `play NAME` | start a game by name |
| `set key=value ...` | change `bm/config.txt` keys until a restart |
| `save` | write `bm/config.txt` |
| `render` , `room` , `log` , `send` , `reboot` | benchmarks, the log as a report, send the waiting reports, reboot |

COPY AND PASTE

```
:set volume=5; save
```

**[TO FILL: shots/boot-os-monitor.png]**

BEWARE

`X` and then `f` freezes the console on purpose, to test the crash screen. Use it only when you want to test that.

# The files on the SD card

The card is FAT32. Names follow the 8.3 rule when the console writes them.

| Path | What it is |
|---|---|
| `bootcode.bin` , `start.elf` , `fixup.dat` | the Pi's firmware |
| `config.txt` (root) | the firmware's configuration |
| `kernel.img` | bm for the Pi Zero W and the Pi 1 |
| `kernel7.img` | the Pi Zero 2 W's kernel, only when built with `ZERO2=1` |
| `kernel8.img` | bm for the RGB30 |
| `carts/` | your cartridges ( `carts/nano8/` for PICO-8 carts) |
| `bm/config.txt` | bm's settings |
| `bm/ca.pem` | certificates for HTTPS |
| `bm/` firmware files | WiFi and Bluetooth chip firmware |
| `bm/save/` | the saves of the games |
| `bm/shots/` | screenshots |
| `bm/reports/` | test and crash reports waiting to be sent |
| `bm/backup/` | the old kernels, kept by an update |
| `bm/lib/` | resource files sent from the PC |
| `bm/bootlog.txt` , `bm/bootprev.txt` , `bm/lastrun.txt` | the RGB30's boot logs (chapter 12) |

**NOTE**

A folder called `bm33` from before the project was renamed is moved into `bm/` by `make install` .

# Settings and config.txt

Do not mix up the two `config.txt`:

- the one in the **root** of the card belongs to the Pi's firmware (chapter 2);
- the one in ** `bm/` ** belongs to bm. It has `key=value` lines. Unknown keys are kept as they are.

Settings, the monitor line `set` / `save` and the PC tools write `bm/config.txt`. These are keys named in the repository:

| Key | What it does |
|---|---|
| `layout` | keyboard layout, `it` or `us` |
| `draw` | `.bm` drawing: `ram` or `direct` |
| `perf` | performance overlay: `off`, `top`, `bottom` |
| `volume` | the volume |
| `sound` | `8bit` or `hifi` |
| `confirm` | `b` or `a`: which button confirms |
| `wifi_ssid`, `wifi_psk` | the network to join |
| `wifi_boot=0` | do not join the network at boot |
| `lib_tab=1` | show the Lib tab |
| `show_bm=0` | hide `.bm` files in the Games tab of the RGB30 |
| `market_bm=1` | list `.bm` files also in the Market of the RGB30 |
| `menu_scale=3` | a 1920x1080 menu on the Pi (a try) |
| `battery_icon=0` | turn off the low-battery icon (RGB30) |
| `mouse=off` | turn the pointer off |

Keys whose name ends in `_token`, `_psk`, `_password` or `_key` are secrets: they are never shown or sent out. Keep keys and tokens only in this file or in the environment, never in other files.

**COPY AND PASTE**

```
wifi_ssid=MyNetwork
wifi_psk=my password
```

## Change a setting from the PC

1. Make sure the console is on the network and note its IP address (Settings > WiFi and network shows it).
2. On the PC run the tool. A line `key=value` sets a key; `key=` removes it.
3. The console answers with the settings as they are now, with the secrets hidden.

COPY AND PASTE

```
./easy_install.sh config
python3 tools/bm_net.py IP --config
```

# Screenshots

Press `Print Screen` on the keyboard, or hold **Select + L1 + R1** on a pad (Share + L1 + R1 on a DS4). The console saves the game's own screen, as it was drawn, as a PNG in `bm/shots/`. The overlays, the battery icon, the pointer and the message box are not in the picture.

## Good to know

- The name is the first letters of the game's title and a number: `PONG0001.PNG`. The number goes up by itself; bm never overwrites a picture. The Pi has no clock, so there is no date.
- A box says `Screenshot saved` and the file name for two seconds.
- The menu's own screens cannot be saved yet. Games and apps can.
- The Print Screen key does not have its picture in the F12 list yet.

## Try it, step by step

1. Start a game from the menu.
2. Press `Print Screen`. A box with `Screenshot saved` appears.
3. Switch the console off and put the card in the computer.
4. Open `bm/shots/`. The picture is there.

> **TRY IT**
> Take three pictures of the same game. You get `0001`, `0002` and `0003` with the same prefix.

**[TO FILL: check Select + L1 + R1 on the PowKiddy RGB30; not tried on the console yet]**

A game takes a picture by itself with `shot()`; see the API manual.

# Saves

A game keeps its data with `save()` and `saved()`. Each game has **8 slots**, each a table of up to 32 KiB, in `bm/save/`.

- Slot 1 is `XXXXXXXX.SAV`; slots 2 to 8 are `.S02` to `.S08`.
- `XXXXXXXX` comes from the title and the author of the game. If you change either, the game starts again from zero.
- A game's options (`c` on the cover) show the bytes used and *Delete the save data* deletes all slots.
- Updates and `make install` never touch settings and saves.

To back up a game's save, copy its files from `bm/save/` to the PC.

# Crashes and logs

## A crash

- A fatal exception shows a red screen with the registers and the list of addresses it was called from, even during a game.
- The last 4 KiB printed and a trail of breadcrumbs stay in memory across a reset. At the next boot they become a report in `bm/reports/`, sent when the network is up. Network and `report()` ask your permission the first time.
- After 20 seconds of running time, the screen says "Restarting in 60 s: this screen goes out as a report", the LED blinks a code, and the console restarts. This also holds on the Pi Zero W: the restart writes the power-management registers directly (`src/kernel/exceptions.c`, `src/drivers/watchdog.c`), so it does not depend on the watchdog check made at boot.
- **[TO FILL: what each LED blink count means on the Pi Zero W; only the RGB30's codes are written down]**

**[TO FILL: shots/boot-os-crash.png]**

A Lua error does not crash the console: the cartridge stops and the error, with its line, appears on screen. A game that runs over 20 million Lua instructions in one frame is stopped too.

## Where the logs are

| Where | What it holds |
|---|---|
| Settings > System > *Log since boot* | everything printed since the console started |
| monitor `o` | the same, scrolling |
| monitor `Z` | the same, sent as a report |
| `bm/bootlog.txt` (RGB30) | the log of this boot, rewritten at several steps |
| `bm/bootprev.txt` (RGB30) | the previous boot's log, how it ended and its last lines |
| `bm/lastrun.txt` (RGB30) | the whole log of a run that ended with a restart or shutdown by bm |

**[TO FILL: a log file on the SD card for the Pi Zero W]**

To read the files, switch off the console, put the card in the PC and open `bm/`. Read them before you switch on again: every boot rewrites `bootlog.txt`.

# Updating

## From the console

1. Connect the console to WiFi (Settings > WiFi and network). You should see an address in the *Address* row.
2. Open Settings > Updates and choose *Check for updates*. It reads the latest signed release and says whether it is newer and which files change.
3. Choose *Install the update*. It downloads and checks every file before writing anything.
4. Wait. The old kernel goes to `bm/backup/` , the board's own kernel is written last, and the console restarts.

If something fails, nothing was written until the files were checked. The monitor key `u` does the same, asking first.

## From the PC

`./easy_install.sh` shows a menu: update the kernel over the network or on the card, a full install, a disk image, send a file, the monitor, the config. The same tool can put an older release back ( `rollback` ).

**COPY AND PASTE**

```
./easy_install.sh kernel
./easy_install.sh install
python3 tools/bm_net.py IP --kernel build/kernel.img
```

**BEWARE**

On the RGB30, if the screen stays black after a restart, read `bm/bootlog.txt` on the PC before you power on again.

# Open points

- **What the kernel does at start**: the boot time of the RGB30 has not been measured in the repository
- **The menu and its tabs**: picture to take: shots/boot-os-menu.png
- **The menu and its tabs**: picture to take: shots/boot-os-settings.png
- **Find out the keys of a game**: picture to take: shots/boot-os-keys.png
- **Find out the keys of a game**: how to see the keys of a game when only a pad is connected; F12 is a keyboard key
- **The monitor**: picture to take: shots/boot-os-monitor.png
- **Screenshots**: check Select + L1 + R1 on the PowKiddy RGB30; not tried on the console yet
- **Crashes and logs**: what each LED blink count means on the Pi Zero W; only the RGB30's codes are written down
- **Crashes and logs**: picture to take: shots/boot-os-crash.png
- **Crashes and logs**: a log file on the SD card for the Pi Zero W